Assignment:

Week 13,14 source code submission, Demo and explain all three tasks with code.

The objective of this project is to design and develop an online chat system, named ClassChat, to be used for communications and discussions among students in a class. The ClassChat is to offer a platform including the function of enabling students to chat with Instructor and other students for any necessary discussions. We give guidelines for the system design of ClassChat.

Task 01: [25 marks]

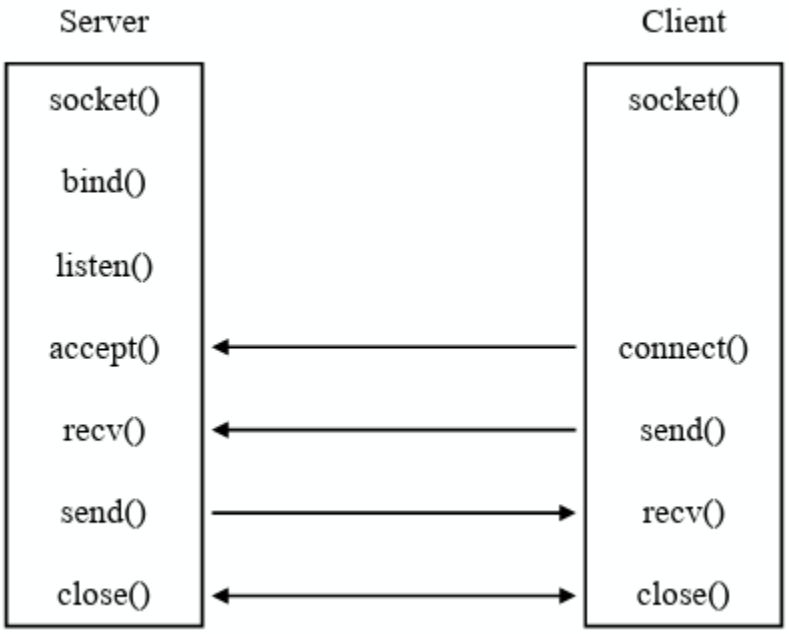
The first step is to build a server and a client that can communicate with each other.

A server should be developed as a central controlling point, which can offer resources and services per clients' request. Here, the server should have a core function of interconnecting the communication of two clients. That is, if a client A wishes to initial a chat with another client b, Both A and B should connect to server, and the server can help forward messages or requests between A and B. To implement a server, the following steps have to be implemented:

- Create a socket for communication
- Bind the local port and connection address
- Configure TCP protocol with port number
- Listen for client connection
- Accept connection from client Send
- Acknowledgment Receive message from client
- Send message to client

To implement a client, the following steps have to be implemented:

- ☐ Create a socket for communication
- ☐ Configure TCP protocol with IP address of server and port number
- ☐ Connect with server through socket
- ☐ Wait for acknowledgement from server
- ☐ Send message to the server
- ☐ Receive message from server



The sketch of client-server communication through socket programming using TCP/IP is shown.

Now, you can implement the communications of a client and a server.

You just use command line to let client and server to communicate with each other. This system can be written with any language, but I strongly recommend Python and Java since they offer convenient tools for socket programming.

Task 02 [35 marks]

After you have implemented the simple client-server communication system, now you can add more function that a client can both send and receive message at the same time with less CPU workload. I/O multiplexing can be used to in this task, in which you can use system callback function to activate a client's application. That is, a client will be activated if the socket receives data from the server or keyboard input from the user. Hint: try to use select(), poll() and epoll() in your client.

Now we would like to improve our client-server communication system by developing a network server to handle multiple concurrent problem. The goal of this task is to allow

multiple students to discuss class topics or homework problem at the same time. There are 3 common ways to implement such function in a server:

- Use socketserver model.
- Thread + socket
- I/O multiplexing

You can select any one method to implement your server.

Task 03: [40 marks]

Now, we are ready to implement the client to client communication in ClassChat.

Your ClassChat should have three core functions:

- Client management.
- Receive message from a sending client.
- Forward message to a receiving client.

The client management is very important at a server. The server should be able to capture the exception that if a receiver is not in the system. For example, A wishes to chat with B, but B is not in the system. Such practice issue should be considered to make your system much robust. Fig. 2 illustrates the main function of ClassChat server. Clients can communicate with each other by passing messages through ClassChat server. Fig. 3 shows a demo of ClassChat. In this figure, Alice and Bob first create connections with the server. Then, Alice sends a message to Bob, the server receives this message and forwards it to Bob.

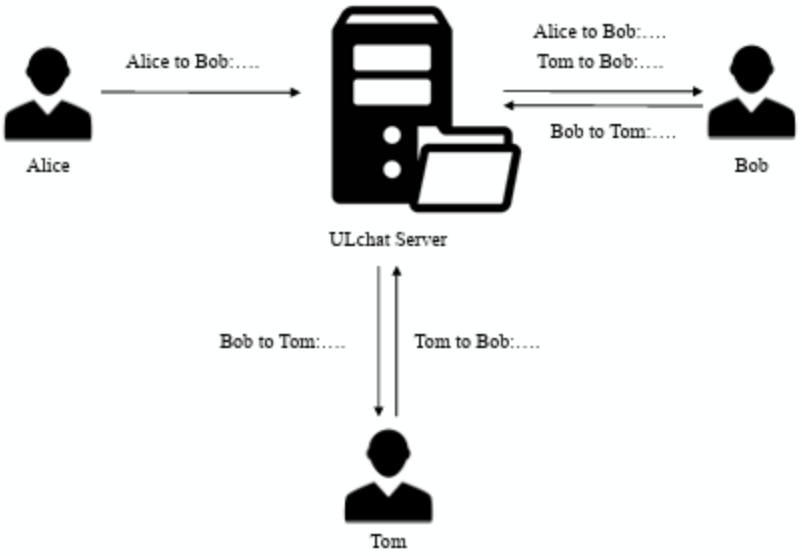


Figure 2: An Flowchart of ClassChat.

```
 $ python client.py
input your user name:Alice
Bob:Hi,do you know how TCP works?
<Alice>:Bob:Hi,do you know how TCP works?
<Bob>: Sure,it is not hard to understand.
```

(a) Client window for Alice

```
 $ python client.py
input your user name:Bob
<Alice>: Hi,do you know how TCP works?
Alice:Sure,it is not hard to understand.
<Bob>:Alice:Sure,it is not hard to understand.
```

(b) Client window for Bob

```
Add new client: Alice
{'Alice': <__main__.MyserverHandler object at 0x10f91c6d8>, 'Bob': <__main__.Mys
erverHandler object at 0x10f91c9e8>}
Send from: Alice to: Bob
Send from: Bob to: Alice
```

(c) Server window

Figure 3: ClassChat demo.

In your client, you should be able to capture sender information, receiver information, and text messages. JSON is a standard format that can be used in network transmission. In this assignment, a simple example like:

```
{
    "status": "1",
    "sender": "Alice",
    "receiver": "Bob",
    "text": "Hi, do you know how TCP works?",
}
```